

# Bringing Wildfire Risk Prediction to the Edge: A TinyML Feasibility Study for Low-Power Sensor Nodes in Algeria

[souhib soltani ]<sup>1</sup>

<sup>1</sup>[Department electronic , University of M'sila, Algeria]

**Abstract** — Machine learning-based wildfire risk prediction is typically evaluated on desktop or cloud hardware, leaving open whether such models can run on the low-power microcontrollers used in real forest sensor deployments. This paper presents a feasibility study for deploying our previously reported Random Forest wildfire classifier (98% accuracy on the Algerian Forest Fires Dataset) on microcontroller-class hardware. We compress the model from 200 to 10 trees (max depth 4), trading 2.1 percentage points of test accuracy (95.9% vs. 98.0%) for a reduction in source size from 295 KB to 5.2 KB. We auto-generate dependency-free C code from the trained model and verify, via an automated equivalence test, that it reproduces the original scikit-learn model's predictions exactly on all 49 held-out test rows — including correctly replicating scikit-learn's soft-voting aggregation, which we found to diverge from a naïve majority-vote implementation on borderline cases. The compiled model occupies approximately 1.4 KB of code space, well within the budget of common microcontrollers such as the ESP32. We demonstrate the model running in an Arduino sketch and a browser-based circuit simulation, and we identify the concrete remaining gaps — on-device Fire Weather Index computation, real sensor integration, and field power-budget validation — required before field deployment.

**Keywords:** *TinyML; edge AI; embedded machine learning; wildfire sensing; Random Forest; microcontrollers; model compression*

## 1. Introduction

Data-driven wildfire risk models are usually validated purely as classifiers — trained and evaluated on desktop hardware, with accuracy as the primary reported metric. This leaves an important practical question unanswered: could the same model realistically run on the resource-constrained microcontrollers used in actual forest sensor networks, where there may be no cellular signal, no continuous power supply, and only kilobytes of memory available for application logic?

This paper is a direct extension of our earlier work, which trained a Random Forest classifier achieving 98.0% accuracy on the Algerian Forest Fires Dataset (Bejaia and Sidi Bel-Abbes regions, 2012). Here we ask: how much of that model — and how much of its accuracy — survives compression to a footprint suitable for a microcontroller, and can we verify correctness rigorously rather than merely observing that the code compiles and runs?

Our contribution is threefold: (1) a systematic accuracy-vs-size trade-off study across Random Forest configurations, identifying a compact 10-tree, depth-4 model retaining 95.9% test accuracy at roughly 1/60th of the original source size; (2) an automatic Python-to-C code generator with a rigorous equivalence test against the original scikit-learn model, which surfaced and let us correct a subtle but consequential implementation bug (naïve majority voting vs. scikit-learn's true soft-voting aggregation); and (3) a working, simulatable demonstration on ESP32-class hardware, together with an honest accounting of what remains before field deployment — most notably, on-device computation of the Fire Weather Index sub-indices that our model relies on as inputs.

## 2. Related Work

TinyML — running machine learning inference directly on microcontrollers — has recently been applied specifically to wildfire monitoring. A 2025 wireless sensor network system built on ESP32 microcontrollers performed real-time fire-risk classification at the network edge from temperature, humidity, and gas sensor data, using a dynamic cluster-head election mechanism for energy balancing, and reported 99% classification accuracy with 25–28 ms inference latency, validating the approach's suitability for remote, resource-constrained deployment. More broadly, TinyML has been applied to environmental monitoring tasks including acoustic anomaly detection and air-quality prediction on low-cost edge devices, and solar-powered multi-sensor environmental stations combining edge inference with low-power wide-area communication (LTE-M/LoRaWAN) have been field-tested for agricultural monitoring.

Our work differs in scope and emphasis: rather than training a new model directly for embedded deployment, we start from a model already validated for scientific accuracy reporting (our companion paper) and treat the embedded port as a separate, independently verifiable engineering step — with an explicit, automated correctness test against the original model, rather than accuracy reported only on the embedded system in isolation. This separation of concerns (scientific validation vs. deployment validation) is, we believe, a useful discipline for reproducible TinyML research.

## 3. Methodology

### 3.1 Model Compression

Starting from the 200-tree, depth-6 Random Forest used in our companion paper, we swept a grid of (n\_estimators, max\_depth) configurations and evaluated each on the same 80/20 stratified split and 5-fold cross-validation used previously. Table 1 summarizes the trade-off.

| Trees         | Depth | Test Accuracy | 5-fold CV | Source Size |
|---------------|-------|---------------|-----------|-------------|
| 200           | 6     | 98.0%         | 98.4%     | ~295 KB     |
| 50            | 5     | 95.9%         | 98.0%     | ~26 KB      |
| 10 (selected) | 4     | 95.9%         | 96.7%     | 5.2 KB      |
| 8             | 3     | 95.9%         | 96.7%     | ~4.1 KB     |
| 5             | 3     | 98.0%         | 96.3%     | ~2.6 KB     |

Table 1. Accuracy vs. size trade-off across Random Forest configurations. All accuracy figures use the identical train/test split as the companion paper.

We selected the 10-tree, depth-4 configuration as a practical default: it retains the same test accuracy as the 8-tree model while leaving more margin in the ensemble (reducing variance from any single tree), at a still-negligible 5.2 KB source size. We note the 5-tree/depth-3 configuration achieves the interesting result of matching the full model's 98.0% test accuracy, but with a visibly lower cross-validation mean (96.3%) and is therefore a less stable choice despite the favorable single-split number — a reminder that model selection under compression should not rely on a single train/test split.

### 3.2 Automatic C Code Generation

We implemented a direct source-to-source translator that walks each scikit-learn DecisionTreeClassifier's internal tree structure (the sklearn ``tree_`` attribute: per-node split feature, threshold, and child pointers) and emits a corresponding C function using nested if/else statements, with no external dependencies. Each per-tree function returns a floating-point class-1 probability, computed from the leaf's training-sample class distribution; the ensemble prediction averages these across all trees and thresholds at 0.5 — a direct C translation of scikit-learn's internal soft-voting aggregation, rather than a simplified hard-majority-vote scheme.

### 3.3 Correctness Validation

An initial implementation used hard per-tree majority voting (each tree contributing one binary vote), which is the more commonly described version of Random Forest inference in embedded-ML tutorials. An automated equivalence test — re-running all 49 held-out test rows through both the Python model and the generated C code — caught a single mismatch, traced to a borderline case where 5 of 10 trees voted for each class under hard voting (a tie, broken toward 'no fire'), while scikit-learn's true soft-voting average (0.544) crossed the 0.5 threshold toward 'fire'. This is a small but consequential discrepancy precisely in the operating region — near the decision boundary — where a wildfire risk system's correctness matters most. After correcting the generator to replicate soft voting exactly, the equivalence test passed with zero mismatches across all 49 test rows.

## 4. Results

The final embedded model compiles to approximately 1.4 KB of code (measured with ``gcc -Os``; a production ARM/Xtensa toolchain is expected to produce a similar or smaller footprint due to more compact instruction encoding), against typical microcontroller budgets of hundreds of kilobytes to several megabytes of flash — leaving

substantial headroom for sensor drivers, communication stacks, and power-management logic. We demonstrate the model running in an Arduino-compatible sketch that cycles through three example scenarios (drawn from real dataset test rows) and drives an LED risk indicator, and we provide a browser-based circuit simulation (Wokwi) requiring no physical hardware to reproduce.

## 5. Discussion and Limitations

**This is an inference feasibility study, not a field-validated system.** We have shown that the trained classifier's decision logic can run correctly and compactly on microcontroller-class hardware. We have not yet built or tested a complete field-deployable sensor node. Three gaps remain, in order of estimated effort:

- (1) **On-device FWI computation.** Our model's strongest predictors (Section 5 of the companion paper) are FFMC and ISI — Fire Weather Index sub-indices computed recursively from raw weather history via the Canadian FWI system's published formulas, not raw sensor readings. Porting this daily recursive calculation to C, with appropriate fixed-point or floating-point precision for a microcontroller, is a well-scoped but non-trivial next step.
- (2) **Real sensor integration.** Temperature, humidity, wind speed, and rainfall sensing are comparatively standard (e.g., BME280, anemometer, tipping-bucket rain gauge) but have not been integrated or calibrated in this study.
- (3) **Field power-budget validation.** The compiled model's tiny footprint suggests inference itself will be a negligible power draw compared to sensing and radio communication, but we have not measured end-to-end power consumption on real hardware under realistic duty cycling, unlike prior work in this space that reports multi-node battery-depletion trends over hundreds of operational cycles.

We view items (1)–(3) as a concrete, appropriately scoped roadmap rather than a weakness to be minimized: each is independently testable and builds directly on the verified inference core presented here.

## 6. Conclusion and Future Work

We showed that a wildfire risk Random Forest classifier can be compressed by roughly 60× in source size while retaining the large majority of its predictive accuracy, and that the resulting model can be translated to dependency-free C and verified — through an automated equivalence test rather than spot-checking — to exactly reproduce the original model's predictions, including a subtle soft-voting aggregation detail that a naïve implementation gets wrong near the decision boundary. Future work will close the three gaps identified above, culminating in a field-deployed, solar-powered sensor node prototype and a follow-up paper reporting real-world power consumption and multi-node reliability, in the spirit of recent TinyML wildfire-monitoring systems.

## Data and Code Availability

All code — the C code generator, the generated model, the equivalence test, and the Arduino/Wokwi demonstration — is available in the *embedded/* directory of the companion repository: <https://github.com/Machine-Learning-Based-Wildfire/Machine-Learning-pipeline-predicts-daily-wildfires-in-Algeria>.

## References

- [Author names withheld for anonymity / to be completed]. Predicting Forest Fire Occurrence in Algeria Using a Random Forest Classifier: A Reproducible Pipeline. Companion paper, this repository.
- Forest Fire Monitoring Using TinyML-Enabled Wireless Sensor Networks. IEEE Conference Publication, 2025. <https://ieeexplore.ieee.org/abstract/document/11203588/>
- Warden, P., & Situnayake, D. (2019). TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers. O'Reilly Media.
- Van Wagner, C. E. (1987). Development and structure of the Canadian Forest Fire Weather Index System. Canadian Forestry Service, Forestry Technical Report 35.

Cost-Effective TinyML-Ready Design and Field Deployment of a Solar-Powered Environmental Monitoring Data Collector Using LTE-M Communication. *Applied Sciences*, 2026. <https://doi.org/10.3390/app16073237>

Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.